Project title: Gesture-Controlled Robotic Arm using Inverse Kinematics

Student Details:

Ritvik Anish – 23BEE1018 – ritvik.anish2023@vitstudent.ac.in

Aneesh Manu – 23BEL1022 - aneesh.manu2023@vitstudent.ac.in

Abstract:

This project presents the design and implementation of a gesture-controlled robotic arm using a software-driven control architecture. The system integrates computer vision-based hand tracking with inverse kinematics to enable intuitive human-robot interaction. Media-Pipe is used to detect hand landmarks in real time, which are mapped to a 3D target position in workspace coordinates. A Damped Least Squares (DLS) inverse kinematics solver computes joint angles required to reach the target. The system also incorporates Jacobian-based control for stable convergence and a finite state machine for task management. A simulation environment using OpenGL was developed to visualize the arm's motion and validate system behaviour. The project demonstrates successful real-time tracking and convergence of the robotic arm to dynamically changing targets. Although full hardware integration was constrained, the modular architecture allows seamless extension to physical robotic systems.

Keywords: Gesture Control, Inverse Kinematics, Damped Least Squares, Jacobian, Robotics, Human-Computer Interaction

Objectives:

- Design a gesture-based control system
- Implement inverse kinematics
- Develop modular robotics architecture
- Simulate robotic arm motion
- Enable intuitive interaction

Existing methods:

Robotic arm control has evolved through several stages, ranging from simple manual systems to advanced vision-based and kinematic control methods. Previously available methods include a traditional robotic arm controlled using manual input - remote-controlled, pre-programmed paths and marker-based tracking [Craig].

Recent developments incorporate inverse kinematics, Jacobian-base control and vision frameworks like OpenGL and MediaPipe [Siciliano][Buss].

Some methods of controlling a traditional robotic arm are using Joysticks, Buttons/Switched and Teach pendants. Such robots don't work in real-time, and hence aren't intelligent systems. They require a pre-programmed set of inputs, like a lookup table or a dataset, and allow joint movements only according to sets defined [Craig]. Any issue that occurs could be either due to robot malfunction and circuitry or due to dataset mismatch. Hence, human intervention is constantly needed and it is difficult to identify how and where the error is. Such systems are not suitable for complex tasks, and are hence not used for automation.

Another traditional method is marker-based tracking, where sensors and cameras are used to detect markers. An example is ArUco Tags, which are mainly used for computer vision and robot navigation. The main limitation of this system is again human intervention. Once markers are found, physical markers are placed at the detected locations to determine the path the robot should follow. Due to the use of high spec cameras, such systems are sensitive to light and are hence not suitable for natural environments [IEEE].

In this project, we try to overcome the main disadvantage of traditional systems – efficiency and human intervention, by using a combination of inverse kinematics, Jacobian and vision-base control strategies. Calculating various parameters, and linking them to the captured gestures shown by the user, hence allowing real-time control with minimal human intervention [Siciliano][Buss]. The user can control the robot from anywhere with an only limitation of camera quality and visibility in order to ensure proper gesture recognition [MediaPipe].

Limitations of existing methods:

- Lack of intuitive control
- Hardware dependency
- Instability in basic IK
- Limited integration

Proposed methodology:

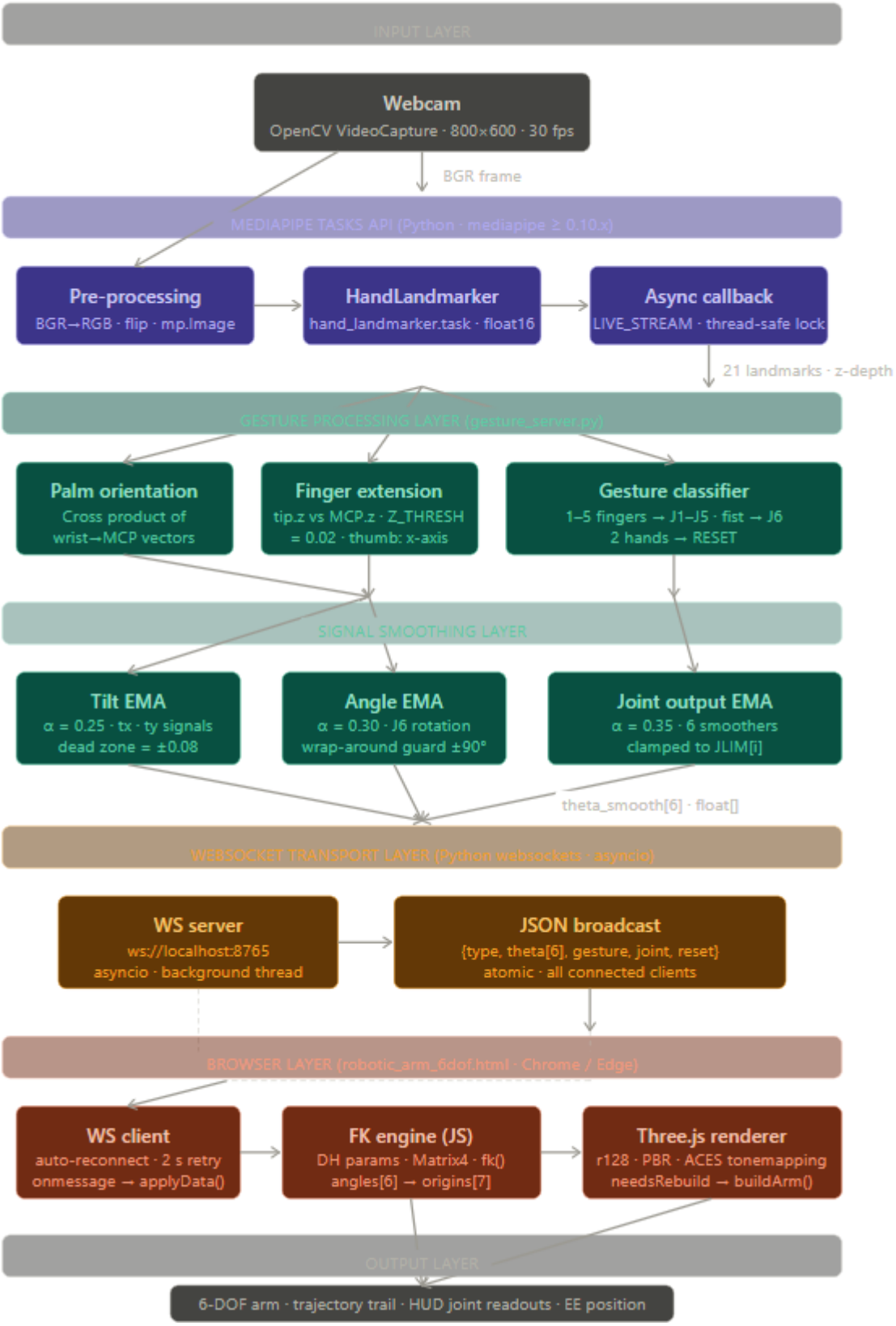
Gesture → Hand Tracking → Target Mapping → IK Solver → Joint Angles → Output

1. Gesture Acquisition: The system begins with capturing user hand gestures using a camera. The gestures act as the primary input for controlling the robotic arm. Unlike traditional methods that rely on physical controllers, this approach allows natural human interaction without external hardware.
2. Hand Tracker: The captured video is processed using MediaPipe, which detects and tracks hand landmarks in real time. The process includes extracting 3D coordinates of fingers and joints, identifying gesture patterns from pre-defined gesture mappings and providing continuous motion data.
3. Target Mapping: Certain gestures and pre-defined, and each gesture map to a joint in the arm. Hand orientation and directions define the joint angle, at which the particular joint rotates. This is how gestures are translated into position and motion commands.
4. Inverse Kinematics Solver: This is the first stage of input processing. It computes joint angles required to make the end-effector reach a position. This should happen in real-time, which means if the object is placed at another position, the robot should be capable enough to recalculate in a split second based on the change in hand gestures.
5. Jacobian-Based Refinement: The next parameter used is the Jacobian Matrix. It related calculated joint angles and joint velocities with the motion of end effector. This is what ensures smooth motion of the arm when it detects a gesture shown by the user.
6. Output and Visualization: The computed values are converted into joint angles for each degree of freedom (DOF) of the robotic arm. Now the created system acts as a continuous feedback loop for real-time operation in 6 DOF.

Model of 6-DOF Robotic Arm used in the project:



Block Diagram:



Hardware components used:

S. No	Component Name	Specification	Purpose
1	Robotic Arm Chassis	6 DOF	Main Body of Model
2	ESP32	5V+Wifi Module	Control of joints and real-time programming of the arm
3	Servo Motors	MG995S, 15kg, 5V	Servos at the joints – rotation and motion of joints
4	Webcam	• Laptop Cam • ESP32 Cam	To detect gestures in real-time
5	Power Supply	Laptop USB or Battery bank	To power 5-6 servos at a time, we require more than just 5V.

Implementation:

- Built kinematic chain
  Designed the structure of the robotic arm by connecting multiple joints and links in sequence. Each joint contributes to the overall motion, forming a complete 6-DOF kinematic chain. This helps in understanding how movement at one joint affects the entire arm.

- Implemented FK and Jacobian
  Implemented forward kinematics to calculate the position of the end effector based on given joint angles. Along with that, I used the Jacobian matrix to understand how changes in joint angles affect the motion of the arm. This was useful for smooth and controlled movement.

- Developed DLS IK
  Developed an inverse kinematics solution using the Damped Least Squares (DLS) method. This helps in avoiding issues like singularities and instability while calculating joint angles. It ensures that the robot moves smoothly towards the desired position.

- Integrated MediaPipe
  Used MediaPipe to detect and track hand gestures in real time. It extracts hand landmarks from the camera input, which are then used to control the robot. This makes the system more interactive and eliminates the need for physical controllers.

- Built OpenGL simulation
  Created a 3D simulation of the robotic arm using OpenGL. This allows visualization of the robot's movement in real time based on the computed joint angles. It helps in testing and understanding the behaviour of the system without actual hardware.

- Software Tools used : Matlab, Python, HTML
  Python was used to implement gesture recognition using MediaPipe and to process hand movements in real time. MATLAB was used to model the robotic arm, perform kinematic computations, and simulate joint movements. HTML (with Web Socket integration) was used to visualize the 3D robotic arm and display real-time motion based on the computed joint angles.

- Data Processing and Collection
  The system collects data using a webcam, where video frames are captured through OpenCV and passed to MediaPipe for real-time hand landmark detection. These landmarks are pre-processed and analysed to extract features such as finger positions, palm orientation, and gesture type. The extracted gesture information is then processed and smoothed using filtering techniques to ensure stable signals. Finally, the processed data is converted into joint angle commands and transmitted via Web Socket for real-time visualization and control of the robotic arm.

Results and Discussion:

Real-time tracking was successfully achieved using continuous hand detection and gesture recognition, allowing the robotic arm to respond instantly to user inputs. The inverse kinematics model showed stable convergence, ensuring that the computed joint angles accurately reached the desired positions without instability. Additionally, the simulation produced smooth motion of the robotic arm, with minimal jerks, resulting in a more realistic and controlled movement.

Novelty of your project:

The project introduces a seamless integration of gesture control with inverse kinematics, enabling intuitive and real-time manipulation of the robotic arm. A stable Damped Least Squares (DLS) approach is implemented to ensure reliable performance even near singular configurations. Additionally, the system is designed in a modular manner, allowing easy integration, scalability, and future enhancements across different components.

- Gesture + IK integration

- Stable DLS approach

- Modular system

Conclusion:

The project demonstrates a gesture-controlled robotic arm using real-time hand tracking and kinematic modelling. By integrating MediaPipe with inverse kinematics and a stable DLS

approach, the system enables smooth and accurate control of joint movements. A modular architecture allows seamless communication between Python, and the web-based simulation. Overall, the system achieves efficient, intuitive control with minimal human intervention and stable performance.

References:

- Siciliano, B., Robotics: Modelling, Planning and Control
- Craig, J.J., Introduction to Robotics
- Buss, S.R., Introduction to Inverse Kinematics
- MediaPipe Hands Documentation
- Relevant IEEE/Elsevier papers on IK and gesture control

Is any dataset used for your project: No

Please indicate your willingness to extend this project work towards Patent Filing or Research Publication: Yes